

Types in Flit

CS 350

Dr. Joseph Eremondi

January 13, 2025

Programming with Holes

Programming with Holes

Why write down types?

- With type inference, we don't *need* to write down types

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check
 - Make sure types in your head match the types in the code

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check
 - Make sure types in your head match the types in the code
- Documentation

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check
 - Make sure types in your head match the types in the code
- Documentation
 - Types make it very clear what arguments a function expects

Checking Types

- Can annotate any expression to confirm its type

Checking Types

- Can annotate any expression to confirm its type

Checking Types

- Can annotate any expression to confirm its type

```
(has-type 3 : Number)  
(has-type #t : Boolean)  
(has-type "hello" : String)
```

Checking Types

- Can annotate any expression to confirm its type

```
(has-type 3 : Number)  
(has-type #t : Boolean)  
(has-type "hello" : String)
```

```
3  
#t  
"hello"
```

Strong Typing

- Flit is *strongly typed*

Strong Typing

- Flit is *strongly typed*
 - Won't do automatic conversions for you

Strong Typing

- Flit is *strongly typed*
 - Won't do automatic conversions for you
 - No `void*` type that we can use to cast through

Strong Typing

- Flit is *strongly typed*
 - Won't do automatic conversions for you
 - No `void*` type that we can use to cast through
 - Doesn't have JavaScript `"1" + 1 = "11"` but `"1" - 1 = 0`

Strong Typing

- Flit is *strongly typed*
 - Won't do automatic conversions for you
 - No `void*` type that we can use to cast through
 - Doesn't have JavaScript `"1" + 1 = "11"` but `"1" - 1 = 0`

Strong Typing

- Flit is *strongly typed*
 - Won't do automatic conversions for you
 - No `void*` type that we can use to cast through
 - Doesn't have JavaScript `"1" + 1 = "11"` but `"1" - 1 = 0`

```
(+ "1" 1)
```

- Gives typecheck failed: Number vs. String

Thinking About Types

- With any type, there are two things we care about

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type
 - How to *directly* use a value of that type

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type
 - How to *directly* use a value of that type
 - Can also create/use any value by giving it to a function expecting/returning that type

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type
 - How to *directly* use a value of that type
 - Can also create/use any value by giving it to a function expecting/returning that type
 - Eventually, you need some primitive operations

- To create: literals 3, 22, 0.00001

- To create: literals 3, 22, 0.00001
- To use: built in arithmetic +, -, *, / etc. and comparisons like zero?

- To create: literals 3, 22, 0.00001
- To use: built in arithmetic +, -, *, / etc. and comparisons like zero?
- Operations have function types

- To create: literals 3, 22, 0.00001
- To use: built in arithmetic +, -, *, / etc. and comparisons like zero?
- Operations have function types
 - + : (Number Number -> Number)

- To create: literals 3, 22, 0.00001
- To use: built in arithmetic +, -, *, / etc. and comparisons like zero?
- Operations have function types
 - + : (Number Number -> Number)
 - zero? : (Number -> Boolean)

- To create: string literals

- To create: string literals
 - `"hello", "", "abcdefg"`

- To create: string literals
 - "hello", "", "abcdefg"
- To use: mostly built-in operations

- To create: string literals
 - "hello", "", "abcdefg"
- To use: mostly built-in operations
 - e.g. `string-ref` : `(String Number -> Char)`

- To create: string literals
 - "hello", "", "abcdefg"
- To use: mostly built-in operations
 - e.g. `string-ref` : `(String Number -> Char)`
 - `string-append` : `(String String -> String)`

- To create: string literals
 - "hello", "", "abcdefg"
- To use: mostly built-in operations
 - e.g. `string-ref` : `(String Number -> Char)`
 - `string-append` : `(String String -> String)`
 - I'll highlight relevant functions as we see them

- To create: literals `#t`, `#f`

Booleans

- To create: literals `#t`, `#f`
- To use: `if someBool e1 e2`

Booleans

- To create: literals `#t`, `#f`
- To use: `if someBool e1 e2`
- E.g. the purpose of booleans is to be things we give to `if` expressions

Types for If

- `if` is not a function

Types for If

- `if` is not a function
 - Doesn't evaluate unused branches

Types for If

- `if` is not a function
 - Doesn't evaluate unused branches
- Need a special rule to determine its type

Types for If

- `if` is not a function
 - Doesn't evaluate unused branches
- Need a special rule to determine its type
 - Can have any type, depending on the type you give it

Types for If

- `if` is not a function
 - Doesn't evaluate unused branches
- Need a special rule to determine its type
 - Can have any type, depending on the type you give it
- Just like we have equations, we can make formal rules for typing

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

then:

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and

then:

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$
- Condition must always be boolean

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

- Condition must always be boolean
- Don't care what the type of the branches is, but the **need to be the same**

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

- Condition must always be boolean
- Don't care what the type of the branches is, but the **need to be the same**
 - Type-checker will give error if branches have different types

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

- Condition must always be boolean
- Don't care what the type of the branches is, but the **need to be the same**
 - Type-checker will give error if branches have different types
- Result of the `if` has the same type as the branches

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
- and $\text{EXPR}_{\text{arg}} : T_1$

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
- and $\text{EXPR}_{\text{arg}} : T_1$
- then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
 - and $\text{EXPR}_{\text{arg}} : T_1$
 - then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$
-
- When asking “how can I make a value of this type”:

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
 - and $\text{EXPR}_{\text{arg}} : T_1$
 - then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$
-
- When asking “how can I make a value of this type”:
 - Can always try “call a function that returns that type”

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
 - and $\text{EXPR}_{\text{arg}} : T_1$
 - then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$
-
- When asking “how can I make a value of this type”:
 - Can always try “call a function that returns that type”
 - The type-checker makes sure that:

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
 - and $\text{EXPR}_{\text{arg}} : T_1$
 - then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$
-
- When asking “how can I make a value of this type”:
 - Can always try “call a function that returns that type”
 - The type-checker makes sure that:
 - The arguments have the type the function expects

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
 - and $\text{EXPR}_{\text{arg}} : T_1$
 - then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$
-
- When asking “how can I make a value of this type”:
 - Can always try “call a function that returns that type”
 - The type-checker makes sure that:
 - The arguments have the type the function expects
 - Multi-argument functions are similar

Function Definitions

Definition (Typing for Function Definitions)

For a definition `(define (f x1 ... xN) body)`

Function Definitions

Definition (Typing for Function Definitions)

For a definition `(define (f x1 ... xN) body)`

- if `body` : `T_ret`

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that $x_1 : T_1 \dots x_N : T_N$*

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
- then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations
 - To *define* function, don't provide things of the argument types

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations
 - To *define* function, don't provide things of the argument types
 - Instead, can pretend you already have things of those types

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations
 - To *define* function, don't provide things of the argument types
 - Instead, can pretend you already have things of those types
 - e.g. the parameters

- Figure out the type of the thing you're trying to write

- Figure out the type of the thing you're trying to write
 - Often, I give this to you

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:
 - Fill in a hole with something of the right type

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:
 - Fill in a hole with something of the right type
 - Which might contain more holes

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:
 - Fill in a hole with something of the right type
 - Which might contain more holes
 - Could be function call, `if`, in-scope variable, etc.

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with 0 or more holes

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with $\emptyset$ or more holes
 - Yay?

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with $\emptyset$ or more holes
 - Yay?
 - Hopefully these holes are simpler

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with 0 or more holes
 - Yay?
 - Hopefully these holes are simpler
 - Can try to fill them with literals, variables, or other function calls

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with $\emptyset$ or more holes
 - Yay?
 - Hopefully these holes are simpler
 - Can try to fill them with literals, variables, or other function calls
- We'll see more of this with recursion and datatypes

Example: Max

Example: Max

```
(define (max [x : Number] [ y : Number]) : Number  
  TODO)
```

x : Number

y : Number

TODO : Number

- We *have* two numbers x and y

Example: Max

```
(define (max [x : Number] [ y : Number]) : Number  
  TODO)
```

x : Number

y : Number

TODO : Number

- We *have* two numbers x and y
- We *want* a number

Example: Max (ctd.)

- Try putting in an `if`

Example: Max (ctd.)

- Try putting in an `if`

Example: Max (ctd.)

- Try putting in an `if`

```
(define (max [x : Number] [ y : Number]) : Number  
  (if TODO TODO TODO))
```

`x : Number`

`y : Number`

`TODO1 : Boolean`

`TODO2 : Number`

`TODO3 : Number`

- Now we need a condition, and something for the true/false cases

- Try putting in an `if`

- Try putting in an `if`

Finishing Max

- Try putting in an `if`

```
(define (max [x : Number] [ y : Number]) : Number  
  (if (<= x y) y x))
```

- The variables `x` and `y` were available for filling the holes

- Try putting in an `if`

```
(define (max [x : Number] [ y : Number]) : Number  
  (if (<= x y) y x))
```

- The variables `x` and `y` were available for filling the holes
 - Because we were inside the function body

- Can use equations to determine the result

<code>(max 3 4)</code>	Starting value
<code>[x=>3][y=>4](if (<= x y) y x)</code>	Sem. of function call
<code>(if (<= 3 4) 4 3)</code>	Do the substitution
<code>(if #t 4 3)</code>	Semantics of <=
<code>4</code>	Semantics of If
